

PROJET OBSILOCK

Cahier des charges

Coffre-fort numérique chiffré — Solution cloud sécurisée

Candidat	BTS SIO option SLAM
Année scolaire	2025 – 2026
Épreuve	E5 – Conception et développement d'applications
Technologies	Java 21 / JavaFX · PHP 8.2 · MySQL 8 · Docker
Compétences mobilisées	B2.1 · B2.2 · B2.3

1. Contexte et présentation du projet

1.1 Présentation de l'organisation

Le projet ObsiLock est développé dans le cadre de la formation BTS SIO option SLAM. Il s'agit d'une réalisation professionnelle complète simulant une commande d'un prestataire informatique souhaitant proposer à ses clients une solution de stockage cloud privé et sécurisé.

Ce projet répond à une problématique réelle : la protection des données sensibles stockées dans le cloud face aux risques de vol, d'interception ou de fuite de données.

1.2 Problématique métier

Les solutions de stockage cloud grand public (Google Drive, Dropbox, etc.) stockent les fichiers en clair côté serveur, exposant les données des utilisateurs en cas de compromission. L'organisation cliente souhaite une solution souveraine dans laquelle les fichiers sont chiffrés de bout en bout, rendant toute interception inutile.

1.3 Rôle dans le projet

En tant que développeur fullstack junior, j'ai participé à la conception de l'architecture, au développement des modules backend et frontend, à la mise en place du système de chiffrement, et à la rédaction de la documentation.

2. Expression des besoins

2.1 Besoins fonctionnels

Authentification et gestion de comptes

- Création de compte utilisateur avec email et mot de passe hashé (bcrypt)
- Authentification sécurisée par tokens JWT (JSON Web Tokens)
- Gestion du quota de stockage par utilisateur — calculé récursivement sur l'ensemble de l'arborescence (racine, dossiers et tous les sous-dossiers)

Gestion de fichiers et dossiers

- Upload et téléchargement de fichiers chiffrés
- Organisation en arborescence de dossiers (hiérarchie récursive illimitée)
- Versionnement automatique des fichiers (historique des versions)
- Téléchargement récursif d'un dossier sous forme d'archive ZIP : le serveur génère le ZIP à la volée en déchiffrant chaque fichier de l'arborescence (dossiers et sous-dossiers inclus) — l'utilisateur reçoit un fichier .zip prêt à décompresser

Partage

- Partage de fichiers ou dossiers via lien tokenisé
- Partage récursif : accès à toute l'arborescence d'un dossier partagé
- Paramètres du partage : date d'expiration, nombre max d'utilisations, label

Interface utilisateur

- Application desktop multiplateforme (Java/JavaFX)
- Explorateur de fichiers intégré (navigation, upload, téléchargement)
- Thème clair / thème sombre avec basculement dynamique

2.2 Besoins non fonctionnels

- Sécurité : chiffrement AEAD XChaCha20-Poly1305 (libsodium) — aucune donnée en clair côté serveur
- Performance : déchiffrement et compression ZIP généré à la volée sans stockage temporaire
- Disponibilité : déploiement Docker (Apache + PHP + MySQL) facilitant la portabilité
- Maintenabilité : code structuré MVC, ORM Medoo côté PHP

2.3 Contraintes juridiques et réglementaires

- Conformité RGPD : les données personnelles (email) sont minimisées — aucune donnée de contenu n'est lisible par l'administrateur
- Droit à l'effacement : suppression complète des fichiers physiques et métadonnées sur demande
- Confidentialité par conception (Privacy by Design) : le chiffrement est structurel, non optionnel

3. Spécifications fonctionnelles

3.1 Cas d'utilisation principaux

Acteur	Action	Résultat attendu
Utilisateur	S'inscrire / Se connecter	Réception d'un token JWT valide 24h
Utilisateur	Uploader un fichier	Fichier chiffré stocké, quota mis à jour
Utilisateur	Télécharger un fichier	Fichier déchiffré restitué au client
Utilisateur	Créer un dossier	Nouveau nœud dans l'arborescence
Utilisateur	Partager un fichier/dossier	Lien tokenisé généré avec paramètres
Utilisateur	Télécharger un dossier	Archive ZIP déchiffrée générée à la volée
Utilisateur	Voir les versions d'un fichier	Historique des versions disponible

3.2 Règles de gestion

- Un fichier ne peut être uploadé que si le quota de l'utilisateur n'est pas dépassé — le quota est calculé en temps réel côté serveur en agrégeant la taille de tous les fichiers sur l'ensemble de l'arborescence (dossiers et sous-dossiers)
- Un partage expiré (date ou nombre d'utilisations atteint) retourne une erreur 410 Gone
- Chaque version d'un fichier possède son propre chiffrement (DEK distincte)
- La suppression d'un fichier entraîne la suppression physique de tous ses blobs chiffrés
- L'arborescence de dossiers est récursive via le champ parent_id (NULL = racine)

4. Architecture et choix techniques

4.1 Architecture générale

Le projet adopte une architecture Client-Serveur découplée :

- Frontend : application desktop Java 21 / JavaFX communiquant avec l'API via HTTP/REST
- Backend : API REST PHP 8.2 (Slim Framework), base de données MySQL 8, ORM Medoo
- Déploiement : conteneurisation Docker (trois conteneurs : Apache/PHP, MySQL, volumes)

4.2 Modèle Conceptuel de Données (MCD simplifié)

Entité	Attributs clés	Relations
Utilisateurs	user_id, email, password (hash), quota_total, quota_used	1 utilisateur → N dossiers
Dossiers	folder_id, user_id, parent_id, name	Récuratif (parent_id = NULL si racine)
Fichiers	file_id, folder_id, filename, size, checksum	1 fichier → N versions
Versions	version_id, file_id, blob_chiffré, nonce, key_envelope, key_nonce, auth_tag	Contient le contenu chiffré
Partages	share_id, token, expires_at, max_uses, label, file_id?, folder_id?	Peut référencer fichier ou dossier

4.3 Architecture de sécurité — Chiffrement

Le projet implémente un chiffrement AEAD XChaCha20-Poly1305 via la bibliothèque libsodium :

- DEK (Data Encryption Key) : clé unique générée par fichier/version
- Key Envelope : la DEK est chiffrée par une clé maître côté serveur avant stockage
- Métadonnées stockées en base : nonce, key_envelope, key_nonce, auth_tag
- Aucun contenu en clair n'est jamais persisté sur le serveur

4.4 Environnement de développement

Élément	Détail
IDE Frontend	IntelliJ IDEA / Scene Builder (fichiers .fxml)
IDE Backend	Visual Studio Code + extensions PHP
Versioning	Git (GitHub)
Déploiement	Docker Desktop (docker-compose)
Tests API	Postman / cURL
BDD	MySQL Workbench / phpMyAdmin

5. Livrables du projet

- Code source backend (PHP/Slim) — accessible via dépôt Git
- Code source frontend (Java/JavaFX) — accessible via dépôt Git
- Scripts SQL de création de base de données

- Fichier docker-compose.yml pour déploiement
- Documentation technique (présente réalisation)
- Documentation utilisateur (guide d'utilisation)
- Rapport de tests fonctionnels

6. Compétences du référentiel BTS SIO SLAM mobilisées

Code	Compétence	Application dans ObsiLock
B2.1	Conception et développement d'une solution applicative	Architecture MVC, développement API REST + client JavaFX, chiffrement libsodium
B2.1	Analyse du besoin et contexte juridique	RGPD, Privacy by Design, droit à l'effacement
B2.1	Exploitation d'un framework	Slim Framework (PHP), JavaFX (Java)
B2.1	Échanges entre applications	API REST JSON, tokens JWT, HTTP client Java
B2.2	Maintenance évolutive	Ajout du versionnement, du partage récursif, du thème dynamique
B2.2	Tests et corrections	Tests Postman, corrections de bugs JWT, gestion des erreurs HTTP
B2.3	Conception d'une base de données	MCD avec héritage (partages), récursivité (dossiers), versions (1:N)
B2.3	Exploitation SQL / ORM	Requêtes via Medoo ORM, gestion du quota en temps réel